QUESTION 1
Type: mcq
Prompt:
Which deployment strategy is best for validating a risky release with a small slice of production traffic first?
Guidance:
Pick the safest progressive delivery method.
Rubric:
Strong answers should minimize blast radius before global rollout.
Solution:
Use a canary release because it validates real traffic gradually.
Allow Multiple: false
Options:
- [ ] Big-bang deploy during off-hours
- [x] Canary release with staged rollout
- [ ] Full blue-green cutover to every user
- [ ] Shadow deploy with no user-facing reads
END QUESTION

QUESTION 2
Type: debug
Prompt:
An API started timing out after a dependency upgrade. Walk through how you would debug it.
Guidance:
Cover telemetry, reproduction, rollback criteria, and how you isolate the change.
Rubric:
Strong answers identify baselines, traces, and mitigation steps.
Solution:
Compare pre/post-upgrade latency, inspect traces, isolate the slow dependency, and define rollback plus instrumentation.
Language: typescript
Starter Code:
```ts
export async function fetchAccount(id: string) {
  const profile = await profileClient.get(id);
  const invoices = await billingClient.listInvoices(id);
  return { profile, invoices };
}
```
Expected Outcome:
Candidate explains where to instrument, what changed, and how to restore stability safely.
END QUESTION

QUESTION 3
Type: scenario
Prompt:
Design the first 48 hours of an engineering sprint for a reliability fix.
Guidance:
Show milestones, owners, and risk controls.
Rubric:

Look for sequencing, execution realism, and communication.
Solution:
An ideal answer includes triage, implementation, observability, validation, and stakeholder updates.
Deliverable: Execution plan
Constraints:
- One backend engineer and one frontend engineer
- Fix must be observable in production metrics
- Stakeholder update due by end of day two
END QUESTION